

Wikipedia Clickstream × Target Stationary Distribution (TSDP)

A Semester Project in Linear Programming and Network Optimization (Research track)

M1 Optimization (2 students / full semester)

September 19, 2025

Abstract

Goal. Given a baseline web-navigation Markov chain P^0 built from the Wikipedia Clickstream dataset and a target stationary distribution π^* , compute a minimally modified transition matrix P such that P is stochastic, respects practical editing constraints, and has stationary distribution π^* . This is the *Target Stationary Distribution Problem* (TSDP) and, with ℓ_1 -type edit costs, it is a linear program (LP) that scales to large graphs.

Why working on this? You will connect optimization to a meaningful application: the *long-run exposure* of Wikipedia pages. You will (i) deal with real data, (ii) model with LPs, (iii) implement scalable solvers (direct LP; column generation is optional), and (iv) produce clear visualizations on before/after navigation dynamics.

1 Background and Data (Wikipedia Clickstream)

Clickstream in one line. Each record counts how often users moved from a **prev** page (referrer) to a **curr** page (resource) during a given month and language. We will use one language (e.g., English) and one month. You will form a directed, weighted graph $G = (V, \Omega, w)$ over article pages V with edges $i \rightarrow j$ and weights w_{ij} equal to monthly transition counts.

Preprocessing.

1. Use only *article*→*article* transitions (type **link**). Optionally keep **other** as a catch-all.
2. Remove non-articles (e.g., portals, special pages) and unify titles after redirects.
3. Choose a *topic subgraph* V (e.g., a category neighborhood around “Machine learning” or “Climate change”). Include the induced edge set $\Omega \subseteq V \times V$ for which you have counts.
4. Build the row-stochastic baseline P^0 : for each $i \in V$ with outflow, set

$$P_{ij}^0 = \frac{w_{ij}}{\sum_{k \in V} w_{ik}} \quad \text{for } j \in V,$$

and handle dangling rows (zero outflow) by either removing such nodes or inserting a small teleport (see Item 3).

Stationary distribution. We adopt the *row-stochastic* convention: $P\mathbf{1} = \mathbf{1}$. The stationary distribution π is a probability vector with $\pi^\top P = \pi^\top$.

Toy Clickstream Subgraph (edge thickness $\propto$ monthly transitions)

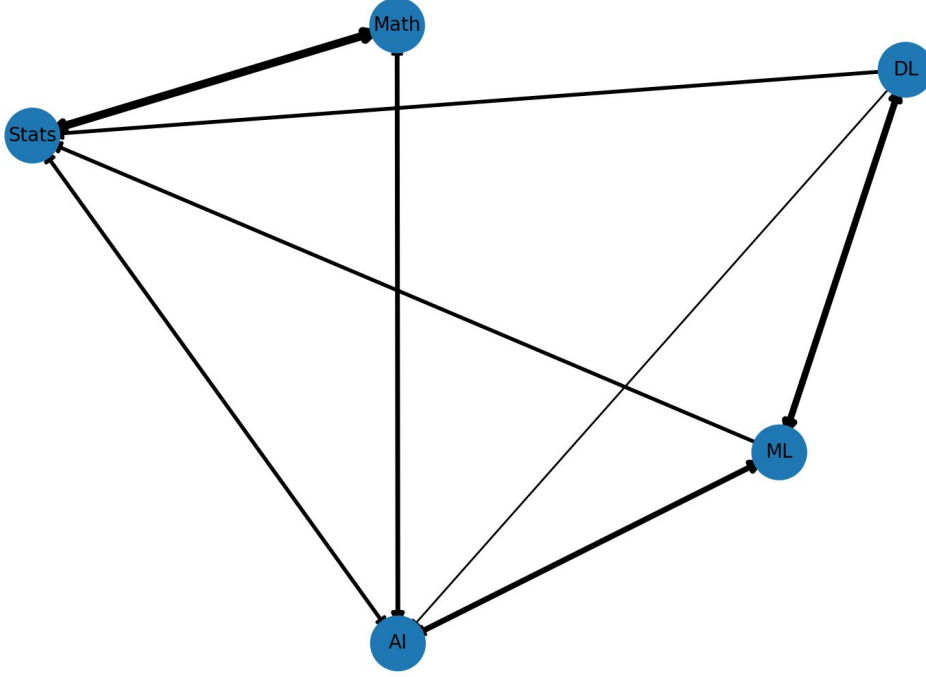


Figure 1: Illustration: Toy clickstream subgraph (edge thickness $\propto$ monthly transitions).

2 Target Stationary Distribution Problem (TSDP)

2.1 Core modeling choices

- **Edit costs.** We want P close to the *teleported baseline* $\tilde{P}^0$ (see Item 3), with per-edge weights $c_{ij} \geq 0$:

$$\text{cost}(P) = \sum_{(i,j) \in \Omega} c_{ij} |P_{ij} - \tilde{P}_{ij}^0|,$$

e.g., $c_{ij} = 1$ (uniform), or higher costs on hard-to-change edges.

- **Target.** The vector $\pi^* \in \mathbb{R}_+^n$ is given (design it in Section 3).
- **Edit support.** Usually, restrict edits to observed edges: $P_{ij} = 0$ for $(i,j) \notin \Omega$. (A relaxed variant allows adding edges from a candidate set.)
- **Stationarity constraint.** Enforce $\pi^{*\top} P = \pi^{*\top}$ (linear in P).

What is proposed below is mentioned in "Main ablations" in Section 9; this allows you to see the impact on the solutions obtained, and gives you the opportunity to deepen your understanding of the problem :) :

1. Variant 1: reversibility (detailed balance; optional) — If you require P to be reversible w.r.t. π^* , add for all i, j :

$$\pi_i^* P_{ij} = \pi_j^* P_{ji}. \tag{1}$$

These are linear equalities in P (since π^* is fixed).

2. Variant 2: ℓ_∞ cost (optional). Minimize the largest per-edge change:

$$\min_{P, \tau} \quad \tau \quad \text{s.t.} \quad \left| P_{ij} - \tilde{P}_{ij}^0 \right| \leq \tau / c_{ij} \quad \forall (i, j) \in \Omega,$$

plus the stochasticity and stationarity constraints; still an LP.

3 Designing the Target π^*

What is π^* ? It is simply the *long-run attention you want per page*. A list of nonnegative numbers that sum to 1.

Two easy ways to specify it (you can do differently if you want!).

1. **Group quotas $\Rightarrow$ page targets.** Partition pages into groups $\{G_1, \dots, G_m\}$ (e.g., subtopics). Choose group shares q_g with $\sum_g q_g = 1$. Distribute each q_g to pages $i \in G_g$ either *proportionally to a baseline* $\tilde{v}_i$ (e.g., $\pi(P^0)$ or outdegree) or *uniformly*:

$$\pi_i^* = q_{g(i)} \cdot \frac{\tilde{v}_i}{\sum_{k \in G_{g(i)}} \tilde{v}_k} \quad \text{or} \quad \pi_i^* = \frac{q_{g(i)}}{|G_{g(i)}|}.$$

2. **Direct per-page targets.** Specify per-page shares explicitly and normalize to sum to 1.

Optional bounds (still linear). Instead of exact π^* , you may impose bounds per page or exact group shares (e.g., $\sum_{i \in G} \pi_i = q_G$) and *also* keep the hard stationarity constraint $\pi^\top P = \pi^\top$ with π in that set (polytopic target).

4 Build the *standard form* $\min c^\top x, Ax = b, x \geq 0$

The first milestone. Formulate the TSDP in the canonical LP standard form

$$\min_x c^\top x \quad \text{s.t.} \quad Ax = b, \quad x \geq 0.$$

5 From Clickstream Counts to P^0 and Quality Checks

1. **Ingest.** Load the monthly TSV (language, month). Keep rows with `type=link`. Parse `prev  $\rightarrow$  curr` and count w_{ij} on your subgraph V .
2. **Normalize.** For each i , set $P_{ij}^0 = w_{ij} / \sum_k w_{ik}$ if $\sum_k w_{ik} > 0$.
3. **Dangling handling and teleport.** First make rows stochastic (replace zero rows by $\nu^\top$), then *define your baseline for optimization* as the teleported matrix

$$\tilde{P}^0 \leftarrow (1 - \alpha) P^0 + \alpha \mathbf{1} \nu^\top, \quad \alpha \in [0.02, 0.10].$$

We recommend uniform ν unless you have a topic prior. *This $\tilde{P}^0$ is the baseline used in the LP objective.* Record (α, ν) .

Implementation note: ensure all rows sum to 1 before teleportation: first replace zero rows with $\nu^\top$, then apply $\tilde{P}^0 \leftarrow (1 - \alpha) P^0 + \alpha \mathbf{1} \nu^\top$ to all rows.

4. **Feasibility sanity check.** Under no-new-edges support, feasibility depends on Ω and π^* . A useful necessary condition: for any page j with $\pi_j^* > 0$, there should exist at least one incoming editable edge $i \rightarrow j$ in Ω (or you must allow adding such edges). If infeasible, relax support or slightly adjust π^* .

Feasibility. Exact stationarity $\pi^{\star\top}P = \pi^{\star\top}$ can be infeasible on very sparse supports. Two robust fixes: (i) optimize against the teleported baseline $\tilde{P}^0$ and allow a small candidate set of extra edges (e.g., self-loops), or (ii) enlarge Ω (higher top- k , broader seeds). Soft stationarity (with small slacks) may be used *only* as a diagnostic.

Data access (practical).

- Download one month from the official Wikimedia dumps: <https://dumps.wikimedia.org/other/clickstream/>.
- Files follow the pattern `clickstream-<lang>wiki-YYYY-MM.tsv.gz`

(e.g., `clickstream-enwiki-2025-03.tsv.gz`).

- Each row has four tab-separated fields `prev`, `curr`, `type`, `n` (see the Clickstream readme¹).
- Keep `type=link` and drop pseudo-pages with titles starting with `other-` (`other-internal`, `other-search`, `other-external`, `other-empty`, `other-other`).
- Pairs with $n \leq 10$ are already filtered out in the public dumps.

If you are stuck with the data pipeline above, you can request a demo code to the instructors at a small cost :)!

6 Algorithms and Implementation

6.1 Direct LP

Implement the standard-form model in Section 4 in Python using `scipy.optimize.linprog` with `method="highs"`².

We strongly advise starting with a small toy example like the one in this document (see Fig. 1 for the graph, and Figs. 2 and 3 for typical results). Once your model and implementation are correct and numerically stable, move to real data from the Wikipedia Clickstream. For this step, use sparse matrices and consider graphs up to $n \approx 1,000$ and $|\Omega| \approx 10,000$.

Practical checklist.

1. Build the arrays $(c, A_{\text{eq}}, b_{\text{eq}}, A_{\text{ub}}, b_{\text{ub}})$ exactly as in Section 4, ordering variables consistently (e.g., stack P_{ij} over $(i, j) \in \Omega$, then slack variables). Use `scipy.sparse.csr_matrix` for $A_{\text{eq}}, A_{\text{ub}}$.
2. Call:

```
res = linprog(c, A_eq=A_eq, b_eq=b_eq, A_ub=A_ub, b_ub=b_ub,
              bounds=bounds, method="highs")
```

3. Verify feasibility and accuracy:

$$\|P\mathbf{1}-\mathbf{1}\|_{\infty} < 10^{-9}, \quad P \geq -10^{-12}, \quad \|\pi(P)-\pi^{\star}\|_1 < 10^{-8} \quad (\text{all w.r.t. the optimized } P; \text{ baseline is } \tilde{P}^0).$$

Treat values in $[-10^{-12}, 0)$ as numerical zero.

¹<https://dumps.wikimedia.org/other/clickstream/readme.html>

²Use of high-level modeling layers such as `cvxpy`, `PuLP`, `Pyomo`, etc., is strictly forbidden. Only call `scipy.optimize.linprog` directly.

4. Report the edit metrics and show diagnostics:

$$\Delta := |P - \tilde{P}^0|, \quad \|\Delta\|_1, \|\Delta\|_\infty, \text{ sparsity} = 1 - \frac{\#\{|\Delta_{ij}| > 10^{-6}\}}{n^2}.$$

Include a bar plot of $\pi(\tilde{P}^0)$ vs. π^* vs. $\pi(P)$ (Fig. 2) and a heatmap of Δ (Fig. 3).

(Optional) Column generation for scaling

For larger instances, a restricted master problem on a subset of edges with iterative column additions can help; see Section 12 for more details and Appendix A for the main routine.

7 Evaluation Protocol and Visualizations

Metrics.

- Edit cost: $\sum c_{ij} |P_{ij} - \tilde{P}_{ij}^0|$ (or ℓ_∞).
- Deviation of *achieved* stationary distribution: $\|\pi(P) - \pi^*\|_1$ (should be ≈ 0 ; useful to detect numerical issues).
- Sparsity / number of edited edges; per-row max change $\max_j |P_{ij} - \tilde{P}_{ij}^0|$.
- Mixing proxy: SLEM (second largest eigenvalue modulus) for reversible P ; for nonreversible P , compare $\|P^k - \mathbf{1}\pi^{*\top}\|$ on a grid of k (specify the norm you use).

Plots

1. **Before/after exposure bars** for top-20 pages: $\pi(\tilde{P}^0)$ vs. π^* vs. $\pi(P)$.
2. **Edge-change network plot** on the topic subgraph: edge width $\propto |P_{ij} - \tilde{P}_{ij}^0|$.
3. **Matrix heatmap** of $\Delta = |P - \tilde{P}^0|$.

8 Illustrative results from instructors

From the toy problem illustrated in 1, we have the typical outputs:

- **Stationary distributions** (bars): $\pi(\tilde{P}^0)$ vs. π^* vs. $\pi(P)$, see Figure 2.
- **Edit map** $\Delta = |P - \tilde{P}^0|$: a sparse heatmap with a small number of nonzero entries; see Figure 3.
- **Numbers**: $\|\Delta\|_1, \|\Delta\|_\infty, \text{sparsity} = 1 - \frac{\#\{|\Delta_{ij}| > 10^{-6}\}}{n^2}$.

9 Experiments to Run

Dataset choices. Pick 1 month and 1 language (e.g., en, de). Build 2 topic subgraphs (one small ~ 100 nodes, one larger ~ 500 – $1k$ (if possible)).

Baselines.

- $\tilde{P}^0$ (teleported baseline).
- (Optional) Reversible construction targeting π^* on the same support.

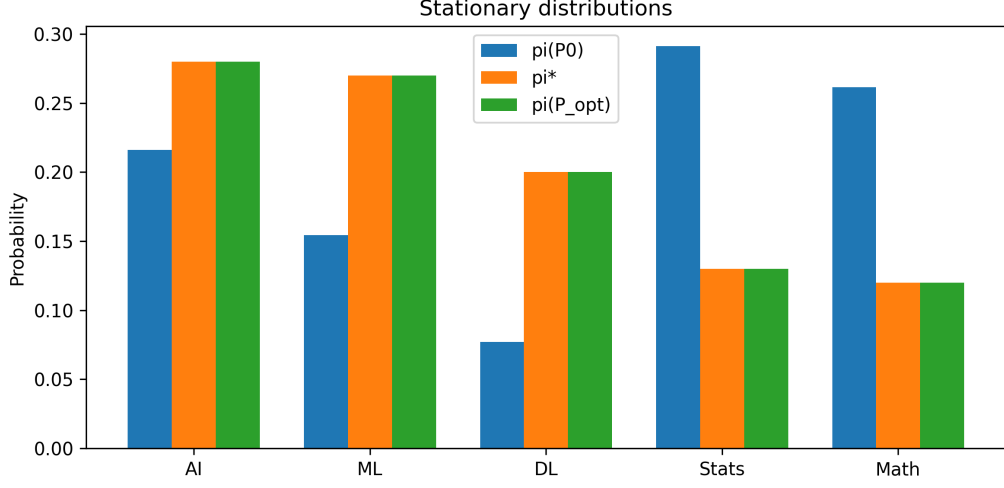


Figure 2: Stationary distributions before/after vs. target (toy instance).

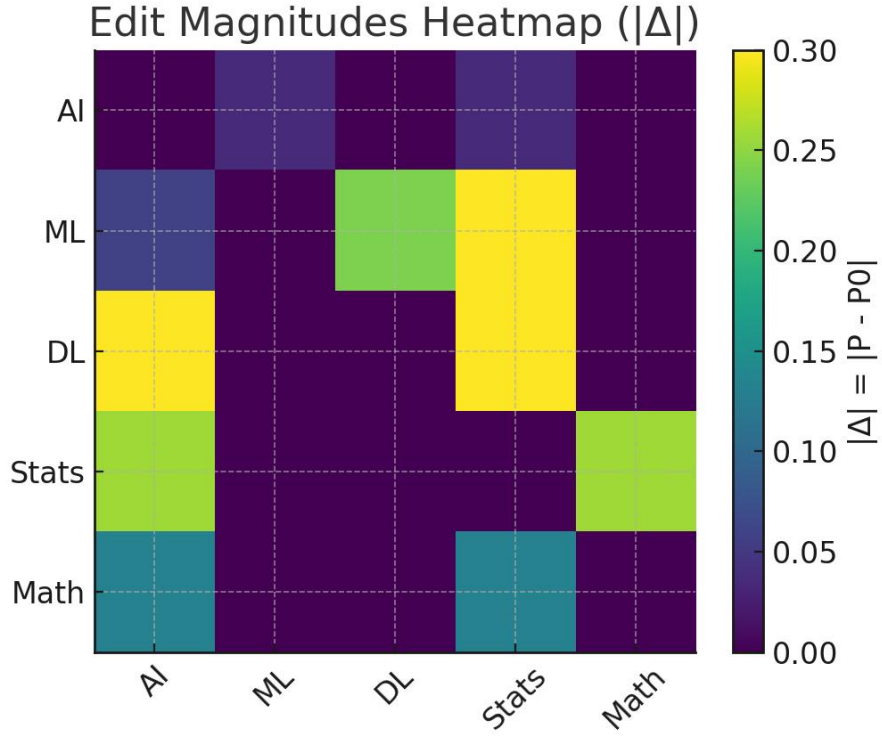


Figure 3: Heatmap of $\Delta = |P - \tilde{P}^0|$ (toy instance).

Main ablations.

1. ℓ_1 vs. ℓ_∞ objective.
2. Uniform vs. degree-weighted edit costs c_{ij} .
3. (Optional) With vs. without reversibility constraints (1).
4. *Column generation vs. direct LP runtime/quality on the larger subgraph (bonus).*

Stress tests. For instance: sensitivity to small perturbations of π^* .

10 Deliverables, Milestones, and Grading

Milestone 1 (first task - midterm). *Standard-form model.* Starting from the modeling choices in Section 4, produce c, A, b and the stacked variable x exactly as in the LP standard form. Provide a short table explaining each block of A .

Final deliverables.

- **Code** (clean, unit-tested): data loader, $P^0/\tilde{P}^0$ builder, LP solver(s), plotting.
- **Final report** (10–20 pages): polished problem statement, LP formulation, standard formulation, data pipeline, full experiments, ablations, limitations, reproducibility checklist.

11 Ethical and Practical Notes

- The Clickstream dataset is aggregated and permissively licensed; still, avoid over-interpreting causality from navigation logs.
- If you use group quotas (exposure targets), clearly justify how groups were defined and avoid sensitive attributes.
- Document all preprocessing choices (filters, redirects, handling of **other-*** referrers, dangling rows).

References

- Wulczyn & Taraborelli (2015). *Wikipedia Clickstream* (monthly aggregated referrer→resource counts).
- Gillis & Van Dooren (2023–2024). *Assigning Stationary Distributions to Sparse Stochastic Matrices* (TSDP; LP + column generation).
- SciPy documentation; HiGHS³ / GLPK manuals.

12 Dual LP & Column Generation (bonus task)

Context and motivation (read this first). The global TSDP can be huge if we allow many potential edges: each candidate transition (i, j) is a decision variable P_{ij} . Rather than building the full LP upfront, *column generation* solves a sequence of smaller LPs. We start with a *restricted master problem* (RMP) on a subset $\Omega_R \subseteq \Omega$ of edges, solve it, read its *dual variables* (α, β, γ) , and then ask: which *missing* edge $(i, j) \in \Omega \setminus \Omega_R$ would most help us reach π^* while paying the smallest ℓ_1 edit? The answer comes from *pricing*: for each missing edge we form the *safe score*

$$s_{ij} = \alpha_i + \pi_i^* \beta_j + c_{ij},$$

which is a sufficient condition for negative reduced cost. If $s_{ij} > 0$, then (by choosing a dual-feasible $\gamma_{ij} = c_{ij}$) adding the variable P_{ij} to the RMP can *decrease* the primal objective (total edit). We then add the best-scoring edges, re-solve, and repeat until no score is positive.

Where the duals come from (one line). α are duals of the row-stochastic constraints $\sum_j P_{ij} = 1$; β are duals of the stationarity constraints $\sum_i \pi_i^* P_{ij} = \pi_j^*$; and $\gamma_{ij} \in [-c_{ij}, c_{ij}]$ arise from the ℓ_1 linearization $|P_{ij} - \tilde{P}_{ij}^0|$.

³<https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs.html>

Reduced costs and a pricing rule

For a candidate edge (i, j) not in the RMP, the reduced cost of P_{ij} is

$$\text{rc}_{ij} = -(\alpha_i + \pi_i^* \beta_j + \gamma_{ij}).$$

Since the dual bound is $-c_{ij} \leq \gamma_{ij} \leq c_{ij}$, the most favorable choice is $\gamma_{ij} = c_{ij}$, yielding the *safe* pricing score

$$s_{ij} = \alpha_i + \pi_i^* \beta_j + c_{ij}.$$

If $s_{ij} > 0$ we can make the reduced cost negative, so add (i, j) .

Mini-task (bonus, step-by-step)

1. **Build the dual** from your standard-form primal. Write it as a compact max problem.
2. **Solve a restricted master LP** on a small editable set Ω_R (e.g., top- k edges per row).
3. **Extract duals** (α, β, γ) from the solver.
4. **Price candidates** $(i, j) \in \Omega \setminus \Omega_R$ using the safe score

$$s_{ij} = \alpha_i + \pi_i^* \beta_j + c_{ij}.$$

Add the top- M edges with $s_{ij} > 0$ to Ω_R .

5. **Repeat** until no $s_{ij} > 0$ (or time budget).
6. **Report** objective vs. time and $\|\Delta\|_1$ vs. iteration on one small graph.

A (Optional) Column generation

Algorithm 1 Sketch: Column Generation for TSDP- ℓ_1 (support-limited)

- 1: Initialize editable set Ω_R (e.g., top- k edges by w_{ij}).
 - 2: **repeat**
 - 3: Solve RMP (LP standard form restricted to Ω_R); get duals (α, β, γ) .
 - 4: Compute scores $s_{ij} = \alpha_i + \pi_i^* \beta_j + c_{ij}$ for $(i, j) \in \Omega \setminus \Omega_R$.
 - 5: Add edges with $s_{ij} > 0$ (most positive first).
 - 6: **until** $s_{ij} \leq 0$ for all candidates or time budget exhausted
-